Taller práctico de Git

Crear una simulación de “página web” para una cafetería llamada Café Aurora. No usaremos código complejo.
El objetivo es aprender Git paso a paso como en un entorno profesional.
Usaremos:

- ramas
- commits
- merge
- stash
- tags
- remotos
- conflictos
- restauración
- historial
- rebase
- etc.

Caso práctico

Tiene esta carpeta: `cafe-aurora/`
Dentro habrá archivos simples:

```
index.html
menu.txt
contacto.txt
README.md
```

PARTE 1 Crear repositorio

1) git init

Inicializa Git.

```
git init
```

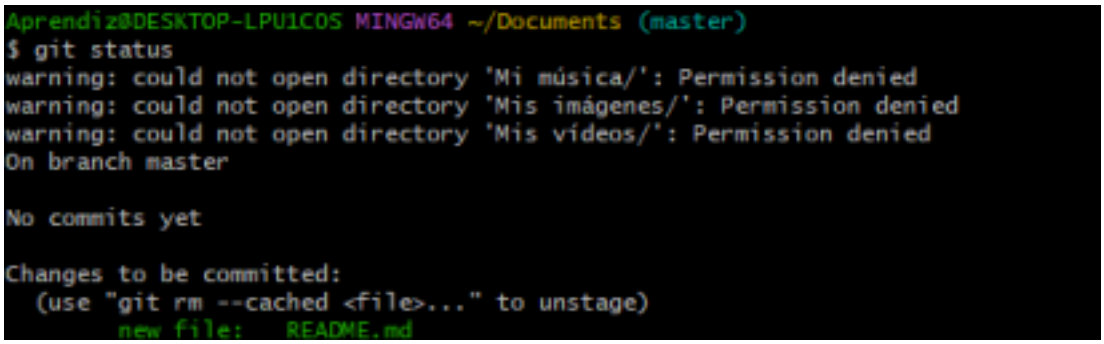
Resultado:

```
Initialized empty Git repository
```

2) git status

Ver estado actual.

```
git status
```



3) git config

Configurar usuario.

```
git config --global user.name "Juan"
git config --global user.email juan@email.com
```

Ver configuración:

```
git config -list
```

PARTE 2 Primer commit

Crea:

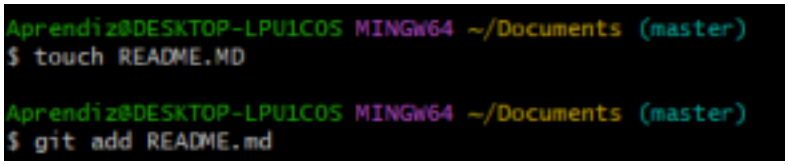
```
README.md
```

Contenido:

```
Proyecto cafetería Cafe Aurora
```

4) git add

```
git add README.md
```

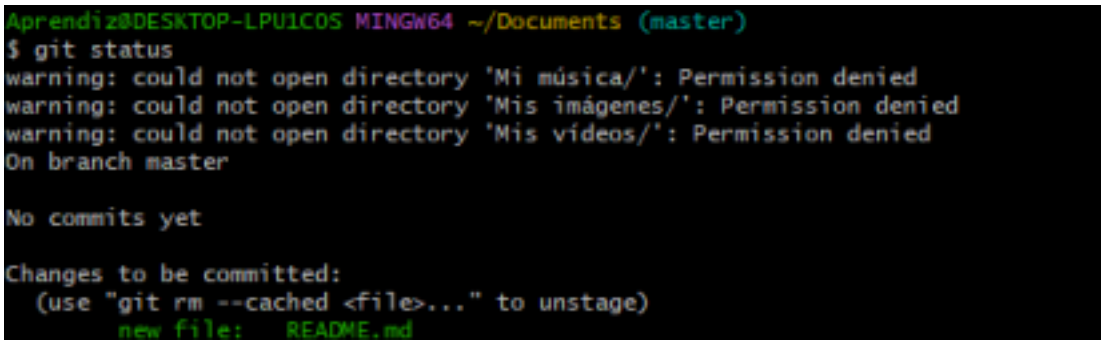


5) git status

```
git status
```

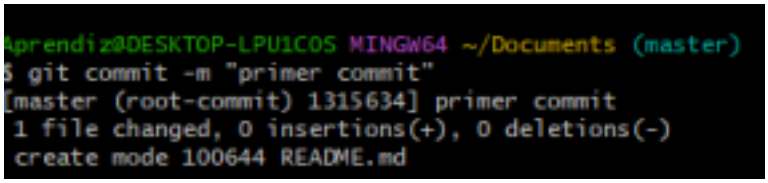
Verás:

```
Changes to be committed
```



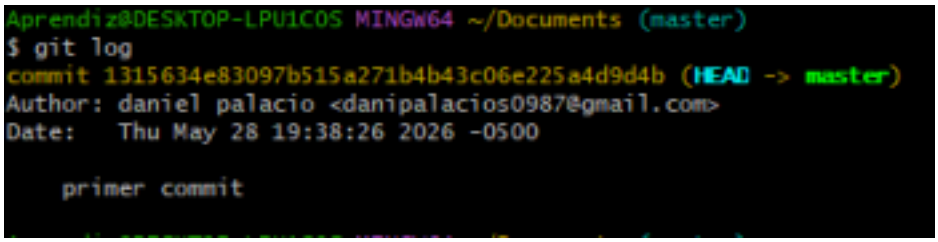
6) git commit

```
git commit -m "Primer commit"
```



7) git log

```
git log
```

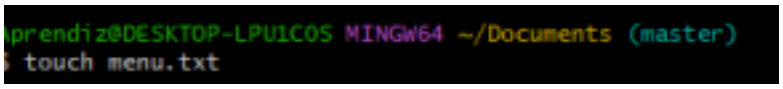


8) git log --oneline

```
git log –oneline
```

PARTE 3 Trabajar con archivos Crea:

`menu.txt`



9) git add .

`git add .`

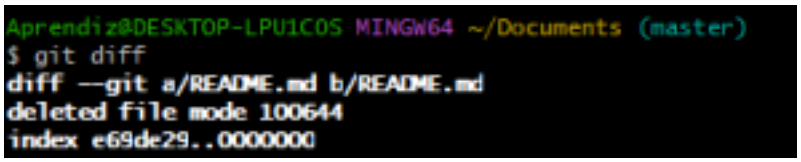
10) git diff

Modifica `menu.txt`.

Luego:

`git diff`

Muestra cambios no staged.



11) git commit

`git commit -m "Agrega menú inicial"`

PARTE 4 Branches

12) git branch

`git Branch`

13) git branch feature/contacto

`git branch feature/contacto`

14) git checkout

`git checkout feature/contacto`

15) git switch

Volver a `main`:

`git switch main`

16) git checkout -b

Crear y cambiar:

`git checkout -b feature/redes-sociales`

PARTE 5 Merge

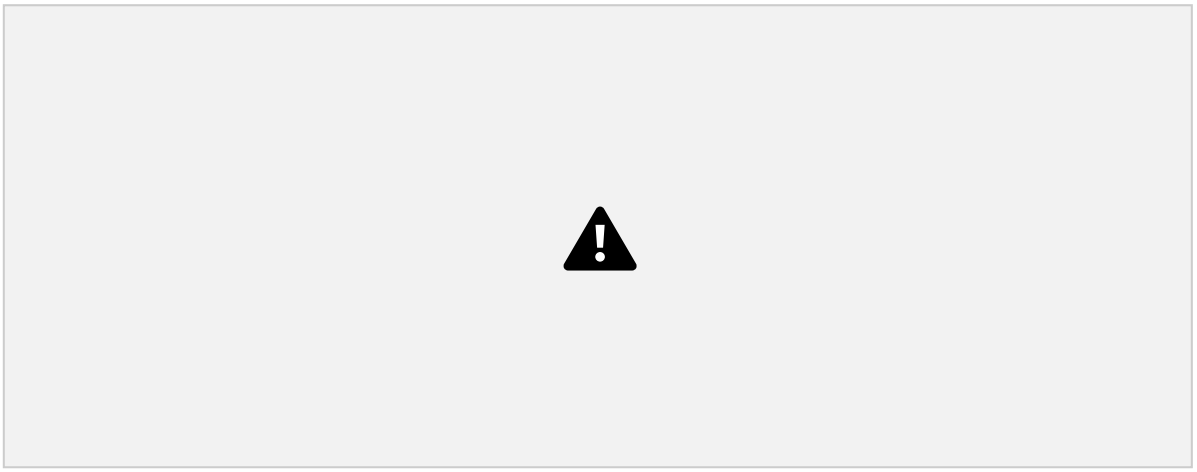
Agrega:

`instagram: @cafeaurora`

Commit:

```
git add .
```

```
git commit -m "Agrega redes sociales"
```



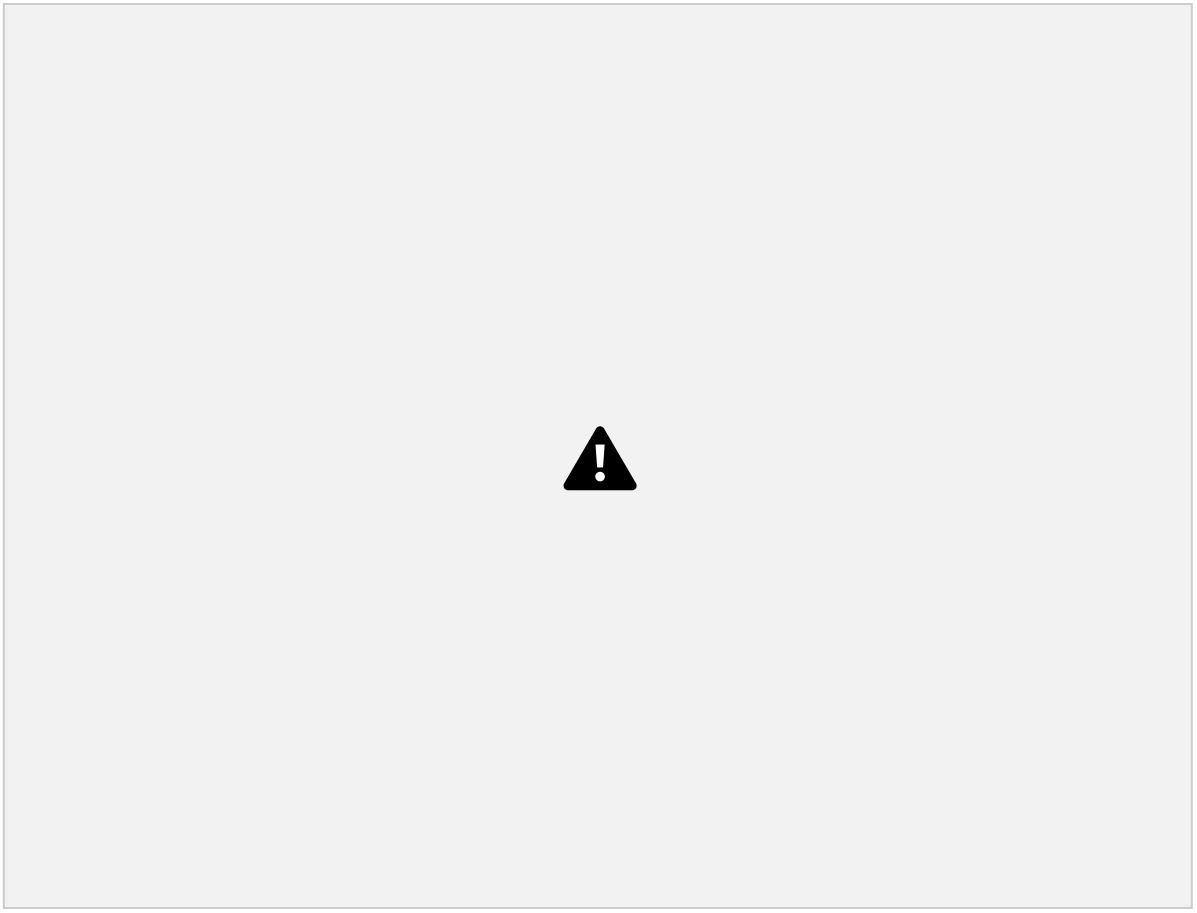
17) git merge

Volver a `main`:

```
git switch main
```

Fusionar:

```
git merge feature/redes-sociales
```



PARTE 6 Eliminar y mover archivos 18) git rm

```
git rm contacto.txt
```

19) git mv

```
git mv menu.txt menu-cafeteria.txt
```

20) git commit

```
git commit -m "Reorganiza archivos"
```

PARTE 7 Restaurar cambios

Modifica `README`.

21) git restore

```
git restore README.md
```

Descarta cambios.

22) git restore --staged

```
git restore --staged README.md
```

Quita del staging.

PARTE 8 Trabajar con remoto (GitHub) 23) git remote add

```
git remote add origin https://github.com/juan/cafe-aurora.git
```

24) git remote -v

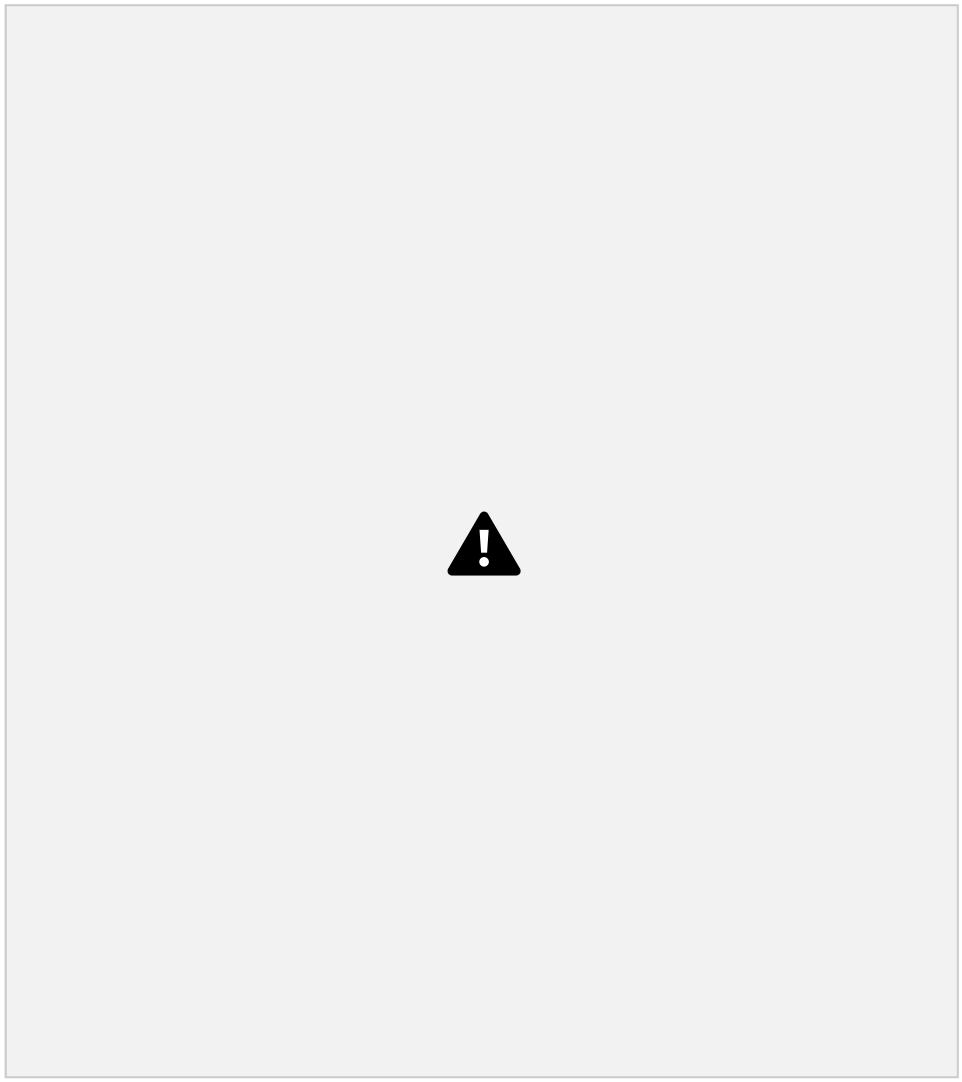
```
git remote -v
```

25) git push

```
git push -u origin main
```

26) git pull

```
git pull
```



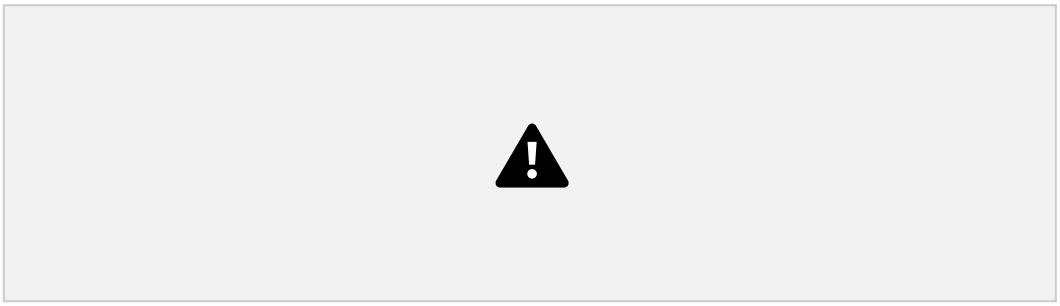
27) git fetch

```
git fetch
```


28) git clone

Otro desarrollador haría:

```
git clone https://github.com/juan/cafe-aurora.git
```



PARTE 9 Stash

Estás trabajando y necesitas cambiar de rama rápido.

29) git stash

```
git stash
```

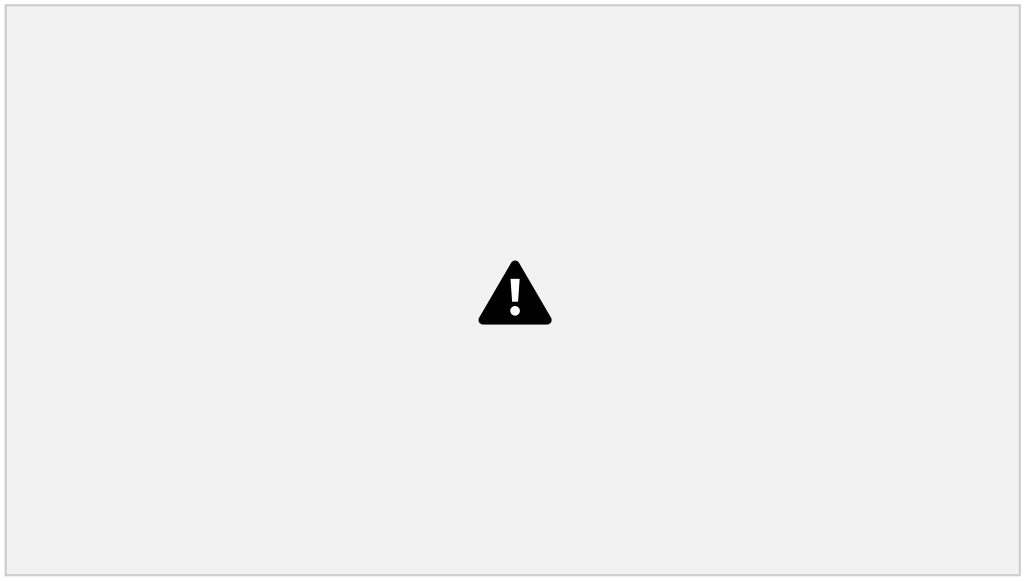
Guarda temporalmente cambios.

30) git stash list

```
git stash list
```

31) git stash pop

```
git stash pop
```



Recupera cambios.

PARTE 10 Historial y detalles

32) git show

```
git show
```

33) git reflog

```
git reflog
```

Historial interno.

34) git blame

```
git blame README.md
```

Quién modificó cada línea

.

PARTE 11 Tags y versiones

35) git tag

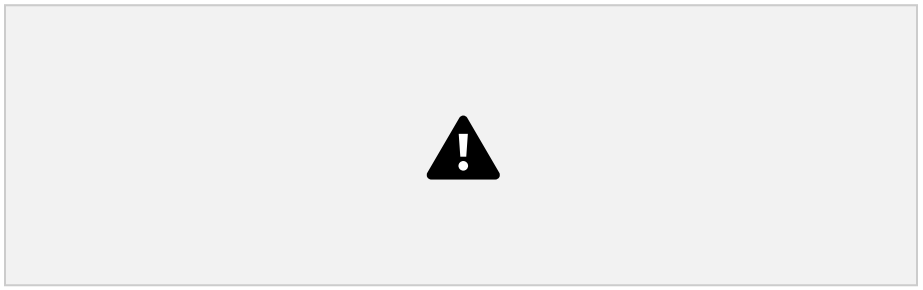
```
git tag v1.0
```

36) git tag -a

```
git tag -a v1.1 -m "Versión estable"
```

37) git push --tags

```
git push –tags
```



PARTE 12

Reset y revert

38) git reset

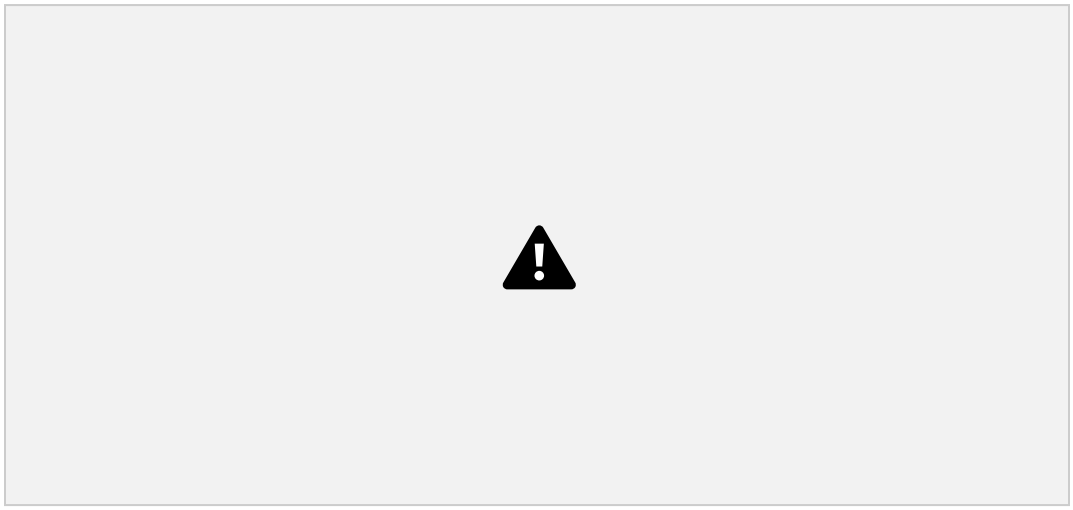
```
git reset HEAD README.md
```

Quitar del staging.

39) git revert

```
git revert HEAD
```

Deshacer commit sin borrar historial.



Ing. Víctor Rincón

PARTE 13 Rebase

40) git rebase

```
git rebase main
```

Reorganiza historial.

EXTRA Flujo profesional

```
git pull
git switch -c feature/menu
git add .
git commit -m "Actualiza menú"
git push origin feature/menu
```

RESUMEN VISUAL

```
Working Directory
 ↓
git add
 ↓
Staging Area
 ↓
git commit
 ↓
Repositorio local
 ↓
git push
 ↓
GitHub
```

LOS 40 COMANDOS APRENDIDOS

1 git init
2 git status
3 git config
4 git add
5 git commit
6 git log
7 git log --oneline
8 git diff
9 git branch

10 git checkout
11 git switch
12 git merge
13 git rm
14 git mv
15 git restore
16 git remote
17 git push
18 git pull
19 git fetch
20 git clone
21 git stash
22 git stash list
23 git stash pop
24 git show
25 git reflog
26 git blame
27 git tag
28 git tag -a
29 git push --tags
30 git reset
31 git revert
32 git rebase
33 git checkout -b
34 git remote add
35 git restore --staged
36 git branch feature/*
37 git switch main
38 git merge feature/*
39 git add .
40 git commit -m

¿Te atreves a realizar el siguiente reto?

Una **simulación real de empresa**, donde usaras:

- comandos básicos (add, commit, branch…)
- comandos intermedios (stash, rebase, reset…)
- comandos avanzados (reflog, bisect, cherry-pick…)

- inspección interna de Git
- herramientas de debugging
- configuración
- limpieza
- flujos de equipo

E-commerce Café Aurora

pro Trabaja en equipo con GitHub. Para desarrollar:

- tienda online
- pagos
- menú
- sistema de usuarios
- corrección de bugs

PARTE 1 CONFIGURACIÓN INICIAL (10 comandos)

```
git --version
git config --global user.name "Juan"
git config --global user.email "juan@email.com"
git config --list
git help
git help commit
git init
git clone https://github.com/cafe/aurora.git
git remote -v
git remote add origin URL
```

PARTE 2 INSPECCIÓN DEL PROYECTO (10 comandos)

```
git status
git status -s
git log
git log --oneline
git log --graph
git log --all
git diff
git diff HEAD
git diff --staged
git show HEAD
```

PARTE 3 WORKFLOW BÁSICO (10 comandos)

```
git add archivo.txt
git add .
git add -A
git commit -m "mensaje"
git commit -am "mensaje"
git rm archivo.txt
git mv viejo.txt nuevo.txt
```

```
git restore archivo.txt
git restore --staged archivo.txt
git clean -f
```

PARTE 4 RAMAS (12 comandos)

Ing. Víctor Rincón

```
git branch
git branch feature/login
git branch -d feature/login
git branch -D feature/login
git checkout main
git checkout -b feature/login
git switch main
git switch -c feature/login
git merge feature/login
git merge --no-ff feature/login
git branch -m nuevo-nombre
git branch -r
```

PARTE 5 REBASE Y HISTORIAL (8 comandos)

```
git rebase main
git rebase -i HEAD~3
git rebase --abort
git rebase --continue
git cherry-pick abc1234
git revert HEAD
git reset HEAD~1
git reset --soft HEAD~1
```

PARTE 6 REMOTO (10 comandos)

```
git push
git push -u origin main
git push origin feature/login
git pull
git fetch
git fetch --all
git remote add origin URL
git remote remove origin
git remote show origin
git ls-remote
```

PARTE 7 STASH (6 comandos)

```
git stash
git stash list
git stash pop
git stash apply
git stash drop
git stash clear
```

PARTE 8 RECUPERACIÓN Y DEBUG (10 comandos)

```
git reflog
git reset --hard HEAD
git reset --mixed HEAD
git reset --soft HEAD
git fsck
git gc
```

```
git blame archivo.txt
git show commit-id
git bisect start
git bisect bad
git bisect Good
```

PARTE 9 VERSIONES (5 comandos)

```
git tag v1.0
git tag -a v1.1 -m "release"
```

Ing. Víctor Rincón

```
git tag
git push --tags
git show v1.0
```

PARTE 10 LIMPIEZA (5 comandos)

```
git clean -n
git clean -f
git clean -fd
git prune
git gc –aggressive
```

PARTE 11 UTILIDADES AVANZADAS (5 comandos)

```
git shortlog
git archive
git describe
git check-ignore
git update-index
```

PARTE 12 AYUDA Y CONTROL (5 comandos)

```
git help
git help merge
git help rebase
git help stash
git config --global -l
```

PARTE 13 FLUJO REAL DE TRABAJO (EQUIPO)

Ejemplo completo de trabajo profesional:

```
git pull
git checkout -b feature/payments
git add .
git commit -m "Add payments system"
git push origin feature/payments
git fetch
git rebase main
git merge feature/payments
git push
```

RESUMEN

En empresas se usa realmente esto:

20% de comandos hacen 80% del trabajo:

- git add
- git commit
- git push
- git pull
- git branch
- git checkout / switch
- git merge

Ing. Víctor Rincón

- git stash
- git log
- git reset

IMPORTANTE

Muchos comandos avanzados como `git bisect`, `git fsck`, `git gc`
Se usan principalmente en:

- debugging
- mantenimiento
- Equipos o proyectos grandes
- recuperación avanzada

CONCLUSIÓN

Git no se aprende por cantidad de comandos, sino por:entender

flujo: `Working Directory → Staging → Commit → Remote`

RETO FINAL

Proyecto: E-commerce Café Aurora Pro

Trabajarás como si estuvieras en una empresa real usando:

- GitHub

- ramas por funcionalidades
- corrección de bugs
- trabajo en equipo
- rebase y merge
- debugging
- recuperación de errores
- versiones estables

Desafío Final

Ing. Víctor Rincón

¿Serías capaz de:

- crear nuevas funcionalidades?
- resolver conflictos?
- recuperar commits eliminados?
- trabajar en equipo sin dañar el proyecto?
- desplegar versiones estables?

Meta del Taller

Al terminar este laboratorio podrás decir:

- “Sé usar Git en proyectos reales.”
- “Entiendo cómo trabajan los equipos de desarrollo.”
- “Puedo colaborar profesionalmente usando Git y GitHub.”

Objetivo del proyecto

Construir una tienda online para:

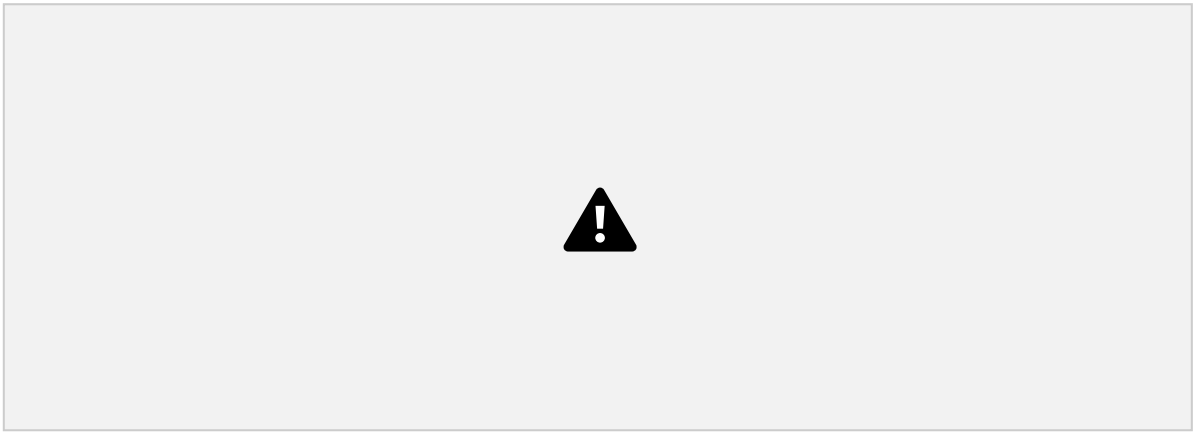
- vender café
- mostrar catálogo
- gestionar pedidos
- manejar usuarios
- integrar pagos
- publicar releases

ARQUITECTURA DEL REPOSITORIO

```
cafe-aurora/
```

```
│
├── frontend/
├── backend/
├── docs/
├── tests/
├── .gitignore
└── README.md
```

Ing. Víctor Rincón



FASE 1 INICIO DEL PROYECTO 1. Clonar o iniciar repo

```
git clone https://github.com/cafe/aurora.git
cd cafe-aurora
```

o si empiezas desde cero:

```
git init
```

2. Configuración del equipo

```
git config --global user.name "Developer 1"
git config --global user.email dev1@cafe.com
```

3. Conectar remoto

```
git remote add origin https://github.com/cafe/aurora.git
```

Ing. Víctor Rincón

FASE 2 PRIMER RELEASE BASE 4. Crear estructura inicial

```
git status
```

5. Agregar todo

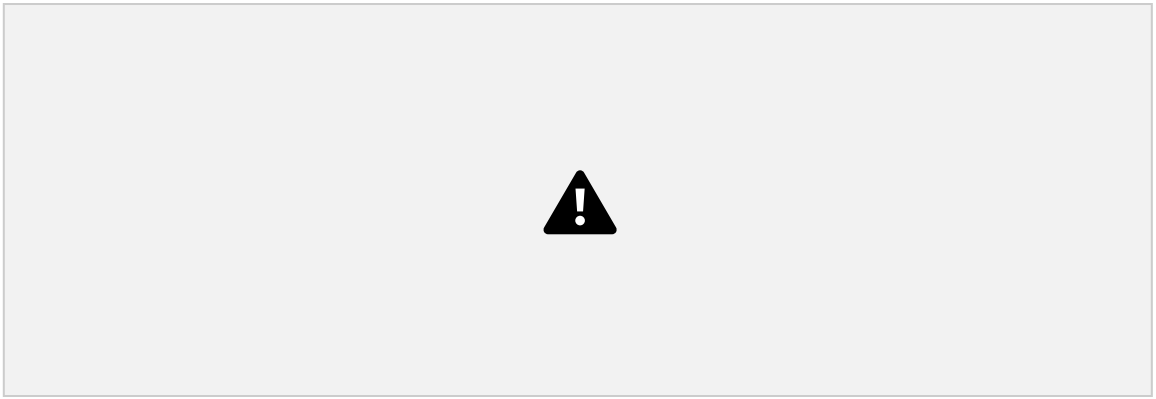
```
git add .
```

6. Primer commit

```
git commit -m "chore: estructura inicial del proyecto"
```

7. Subir a GitHub

```
git push -u origin main
```



Ing. Víctor Rincón

FASE 3 DESARROLLO POR RAMAS

8. Crear rama de desarrollo

```
git checkout -b develop
```

9. Crear feature catálogo

```
git checkout -b feature/catalogo
```

10. Trabajar catálogo

Se agregan productos:

- café colombiano
- café espresso
- café premium
-

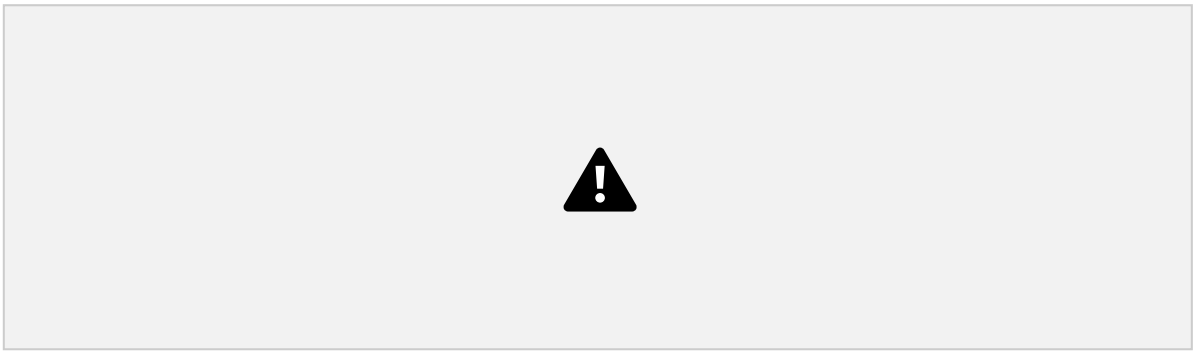
11. Guardar cambios

```
git add .
git commit -m "feat: catálogo de productos inicial"
```

12. Subir rama

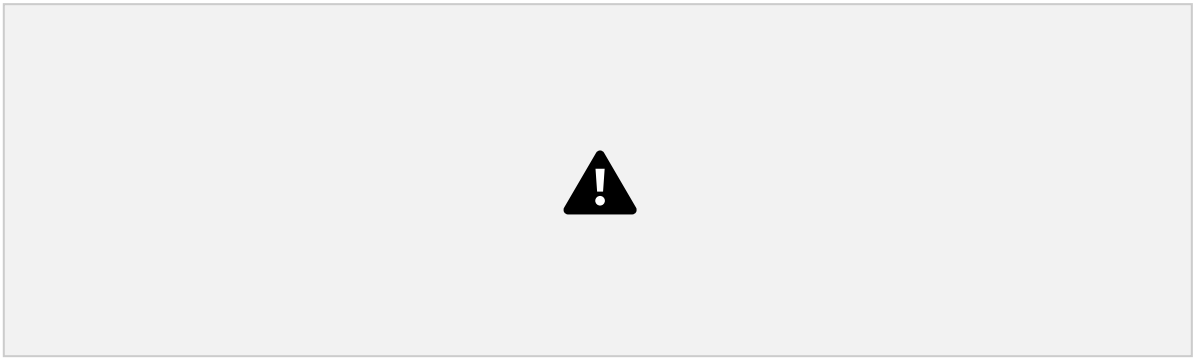
```
git push origin feature/catalogo
```

Ing. Víctor Rincón



13. Merge a develop

```
git checkout develop
git merge feature/catalogo
```



FASE 4 SISTEMA DE PAGOS

14. Crear rama

```
git checkout -b feature/pagos
```

15. Cambios en pagos

Se agregan:

- PayPal
- tarjeta

- Stripe
-

16. Guardar cambios

```
git add .
git commit -m "feat: integración de pagos"
```

17. Subir rama

```
git push origin feature/pagos
```

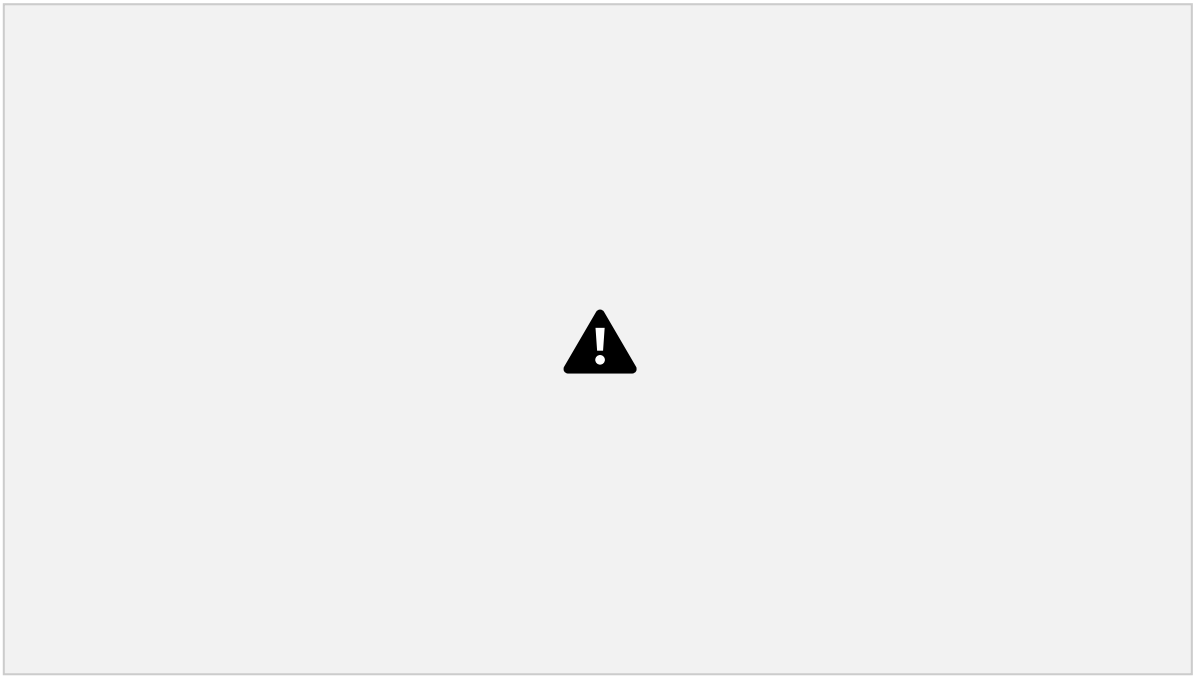
FASE 5 CONFLICTO REAL (IMPORTANTE)

Dos personas modifican el mismo archivo:

`checkout.html`

18. Merge conflict

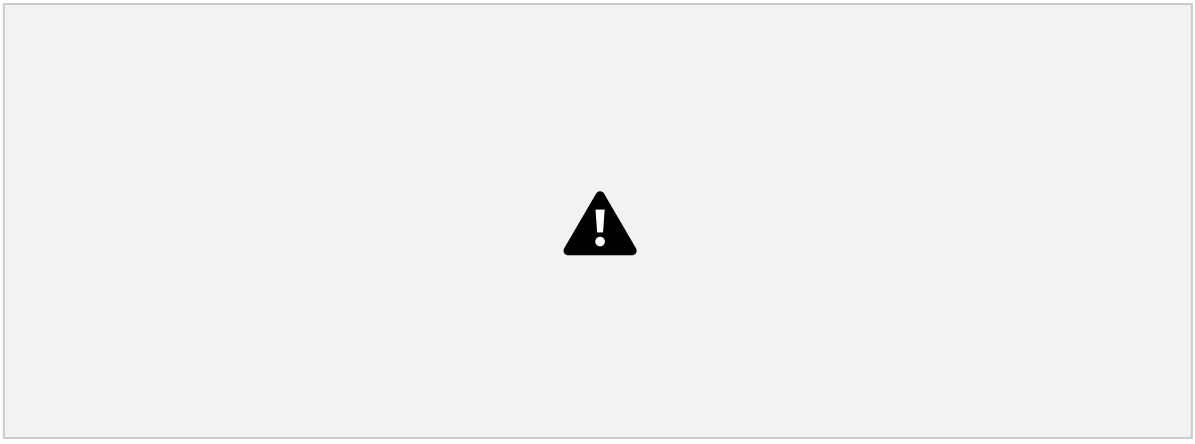
```
git checkout develop
git merge feature/pagos
```

Conflicto aparece.

19. Ver estado

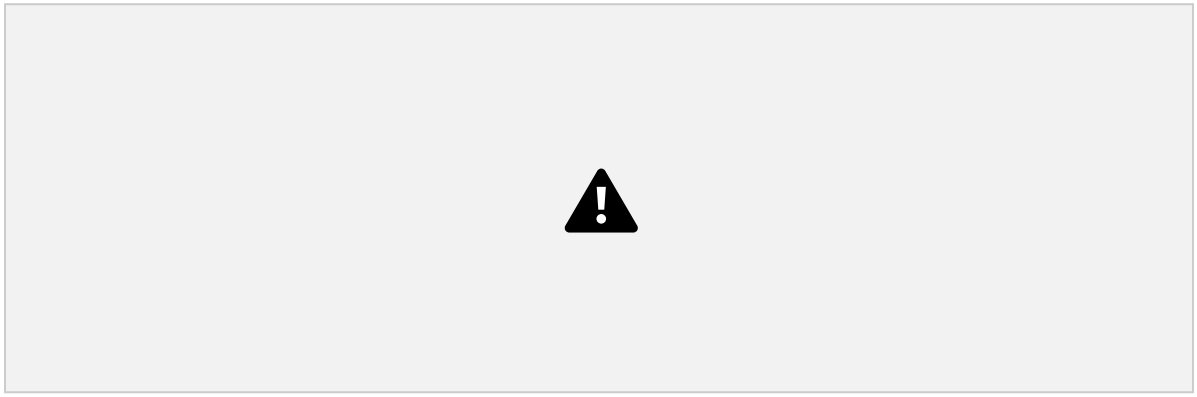
```
git status
```

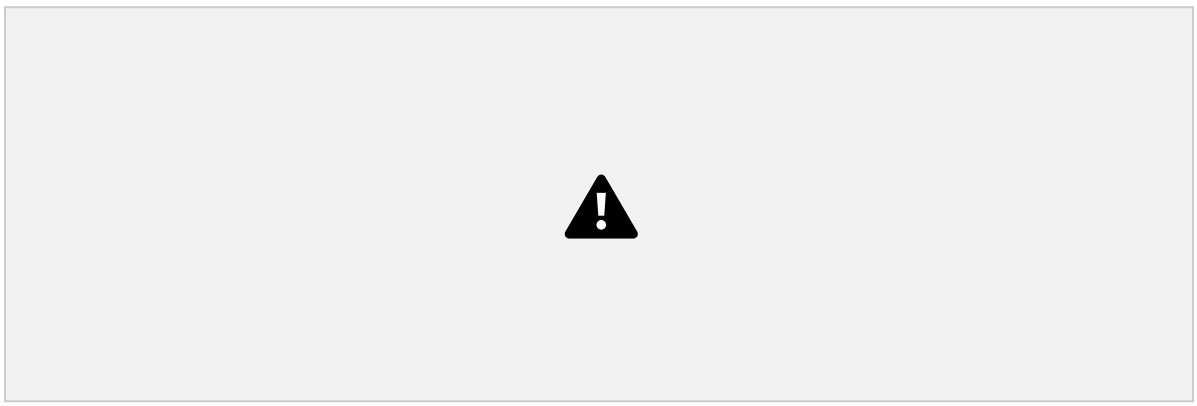


20. Resolver conflicto manualmente

Editas archivo y decides versión correcta.

Ing. Víctor Rincón





21. Marcar como resuelto

```
git add checkout.html
```

```
git commit -m "fix: resolve merge conflict en pagos"
```

FASE 6 RELEASE

22. Crear rama release

```
git checkout -b release/v1.0
```

Ing. Víctor Rincón

23. Ajustes finales

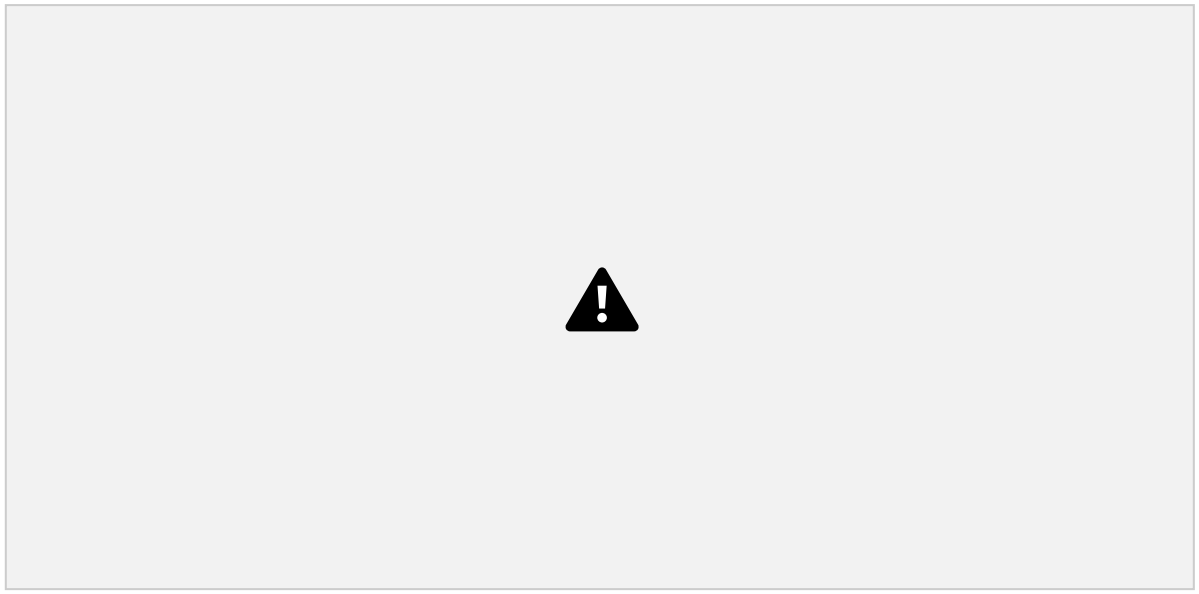
- bugs menores
- UI
- performance

24. Commit final

```
git commit -am "release: versión 1.0 lista"
```

25. Merge a main

```
git checkout main
git merge release/v1.0
```



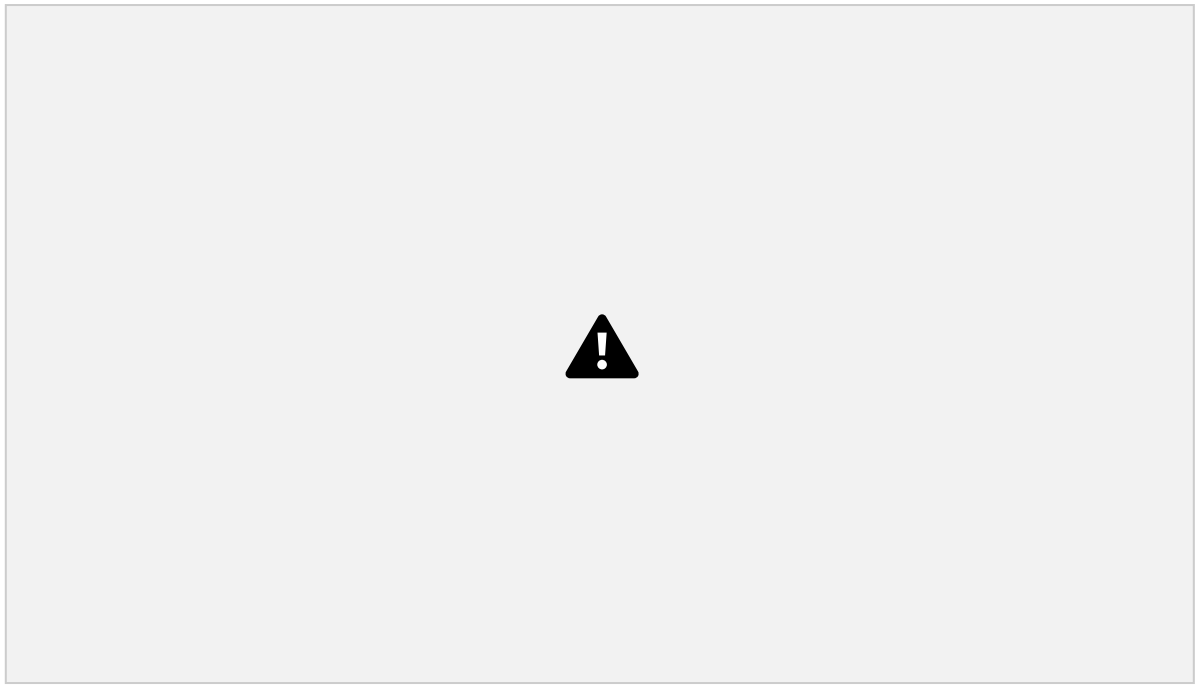
26. Tag de versión

```
git tag -a v1.0 -m "Primera versión estable"
```

27. Subir todo

Ing. Víctor Rincón

```
git push origin main
git push –tags
```



FASE 7 TRABAJO DE EMERGENCIA (HOTFIX)

28. Bug crítico en pagos

```
git checkout -b hotfix/pagos-error
```

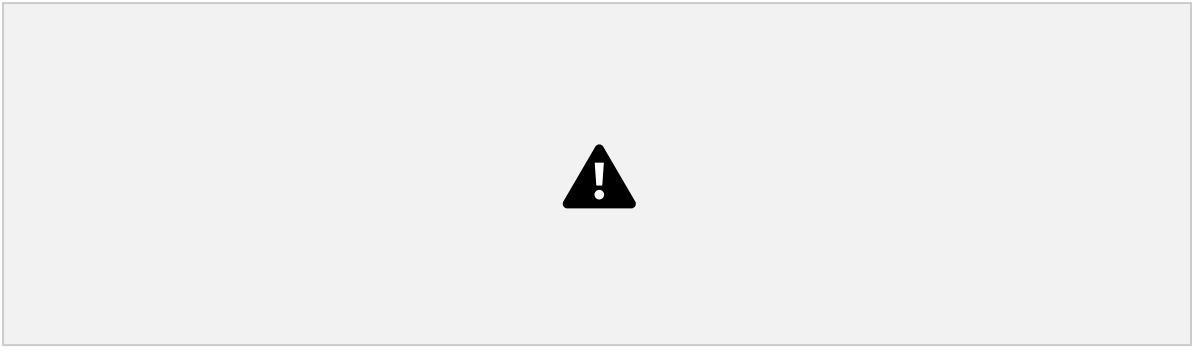
29. Fix rápido

```
git commit -am "hotfix: error crítico en checkout"
```

Ing. Víctor Rincón

30. Merge inmediato

```
git checkout main
git merge hotfix/pagos-error
```



FASE 8 HISTORIAL Y CONTROL

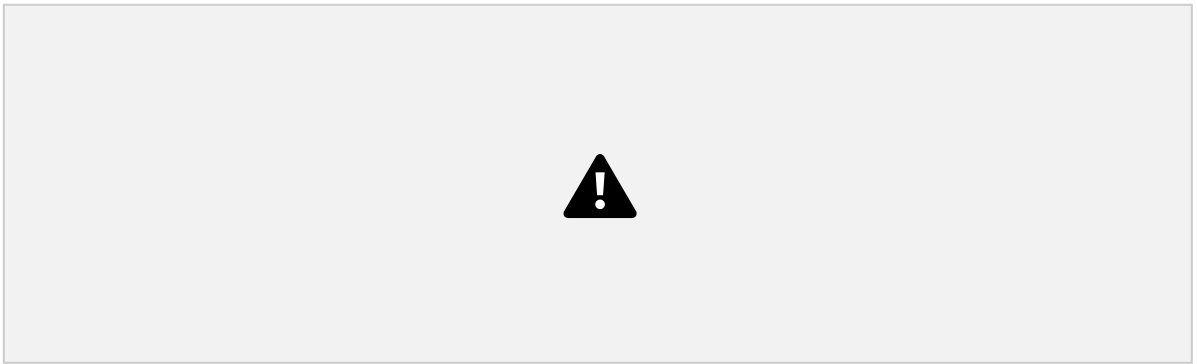
31. Ver historial

```
git log --oneline -graph
```

Ing. Víctor Rincón

32. Ver cambios de un commit

```
git show HEAD
```

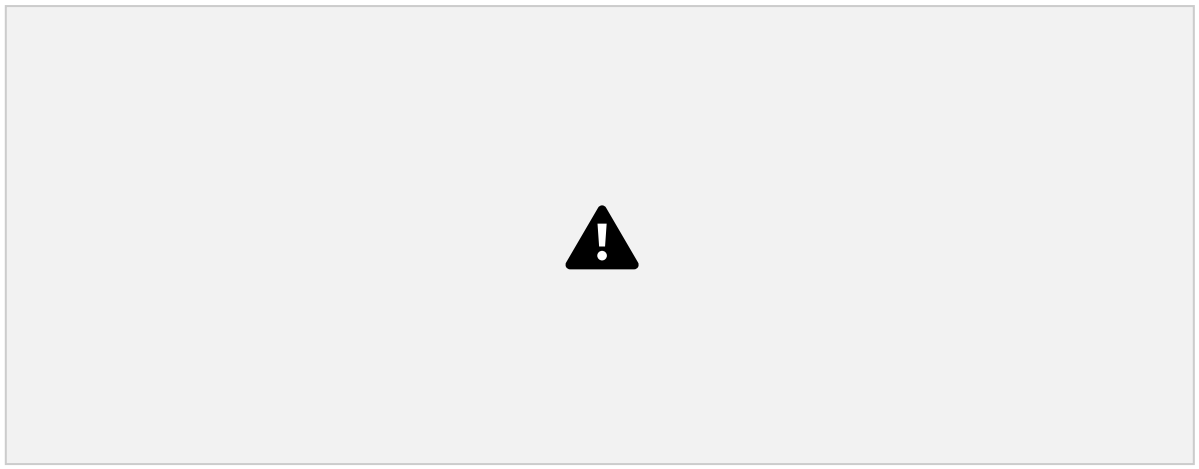


33. Quién modificó qué

```
git blame backend/payments.js
```

34. Recuperar estado anterior

```
git reflog
```



FASE 9 STASH (trabajo urgente)

Ing. Víctor Rincón

35. Guardar cambios temporales

```
git stash
```

36. Volver cambios

```
git stash pop
```

FASE 10 REBASE (historial limpio) 37.

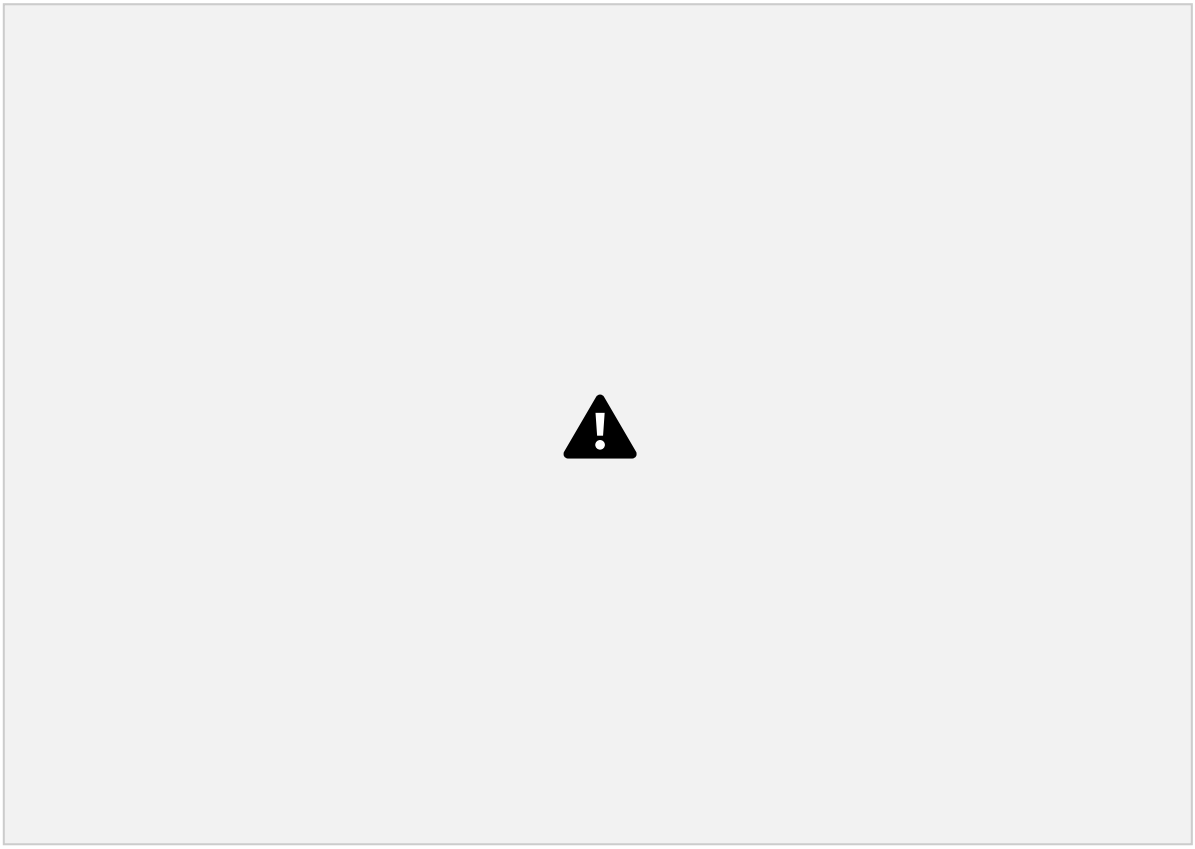
Reorganizar commits

```
git rebase main
```

38. Rebase interactivo

```
git rebase -i HEAD~3
```

Ing. Víctor Rincón



FASE 11 DEBUG AVANZADO

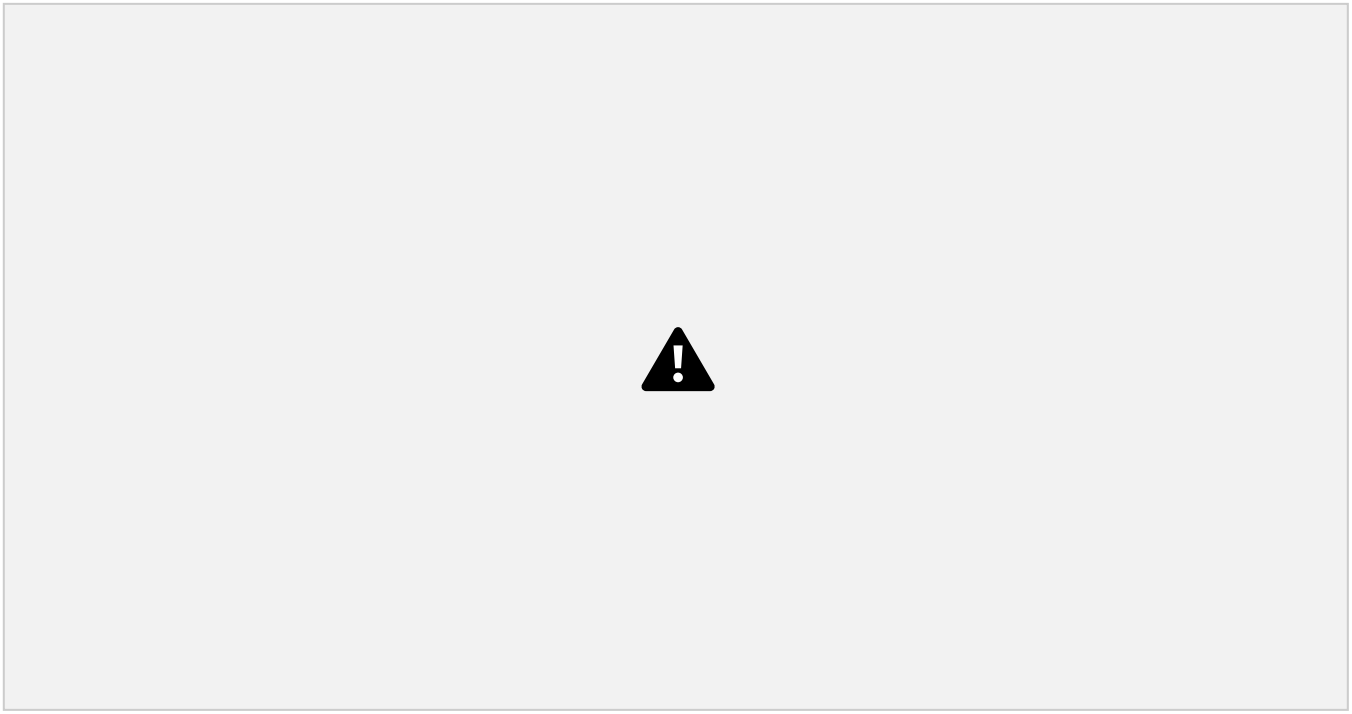
39. Buscar bug

```
git bisect start
git bisect bad
git bisect good v0.9
```

40. Limpiar repo

Ing. Víctor Rincón

```
git gc
git fsck
```



FLUJO FINAL DEL PROYECTO

```
feature → develop → release → main → tag
 ↓ ↓ ↓ ↓
 testing staging producción versión
```

LO QUE APRENDISTE REALIZANDO EL ANTERIOR PROYECTO

Flujo real de empresa

- feature branches
- develop branch
- release branch
- hotfix branch

Trabajo en equipo

- merges

Ing. Víctor Rincón

- conflictos

- rebase

Producción real

- tags
- versiones
- hotfix

Control total

- logs
- reflog
- blame
- bisect